SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	**Assessment:**	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	**Total Marks:**	2 * 50= 100
CO4 Statement:	*Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.*(BL3, BL4,BL5)		
Student Name:	C Anu Keerthana	**Reg. No.:**	192472057
Section / Batch:	CSE-AI	**Date:**	21-08-2026

Instructions to Students

- **Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.**
- Implement the solution using **Python**.
- Select and justify an appropriate AI/LLM model or technique.
- Develop a **working application with a user interface**.
- Test the application using multiple input cases.
- Demonstrate handling of invalid, empty, or unexpected inputs.
- Clearly explain the application architecture and workflow.
- Submit the source code, test results, screenshots/demo, and documentation.
- Explain the limitations of the selected model/technique and suggest possible improvements.

Code of Conduct

I N Goutham/192472024 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N Goutham

SECTION A – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

28. English-to-Tamil Translation Application – Develop a machine translation application that translates English text into Tamil. Provide a user-friendly interface and demonstrate the system using different types of sentences.

1. Title

English-to-Tamil Translation Application Using Generative AI

2. Problem Statement and Objective

Language differences can create communication difficulties for users who are not familiar with a particular language. Machine translation systems help overcome this problem by converting text from one language into another.

The objective of this project is to develop an AI-based application that translates English text into natural and grammatically correct Tamil. The application should provide a simple user interface and support different types of sentences.

3. Technologies and AI Model Used

- **Programming Language:** Python
- **Development Platform:** Google Colab
- **User Interface:** Gradio
- **AI/LLM Model:** Gemini
- **Technique:** Generative AI and Machine Translation

Why Gemini?

Gemini was selected because it can understand the meaning and context of English text and generate natural Tamil translations. It can handle simple sentences, questions, instructions, and longer text while preserving the original meaning.

4. Application Architecture

User
↓
Gradio User Interface
↓
English Text Input
↓
Python Translation Function
↓
Gemini API
↓
Gemini AI Model
↓
Tamil Translation
↓
Display Result

The user enters English text through the Gradio interface. The application validates the input and sends it to the Gemini AI model through the API. Gemini processes the English text and generates a natural Tamil translation, which is displayed to the user.

5. Working Process

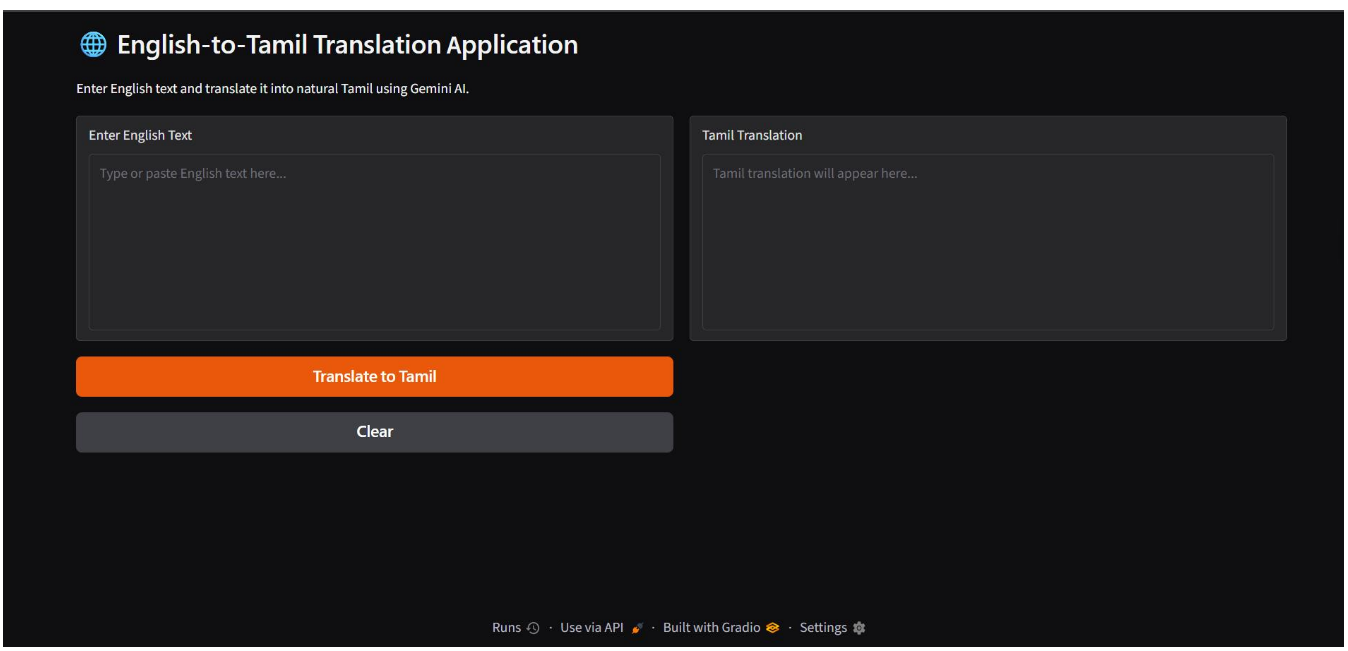
The application works in the following steps:

1. The user enters English text.
2. The application checks whether the input is valid.
3. The English text is sent to the Gemini AI model.
4. Gemini analyzes the meaning and context of the text.
5. The model generates the Tamil translation.
6. The translated text is displayed in the output section.

6. Application Features

The main features of the application are:

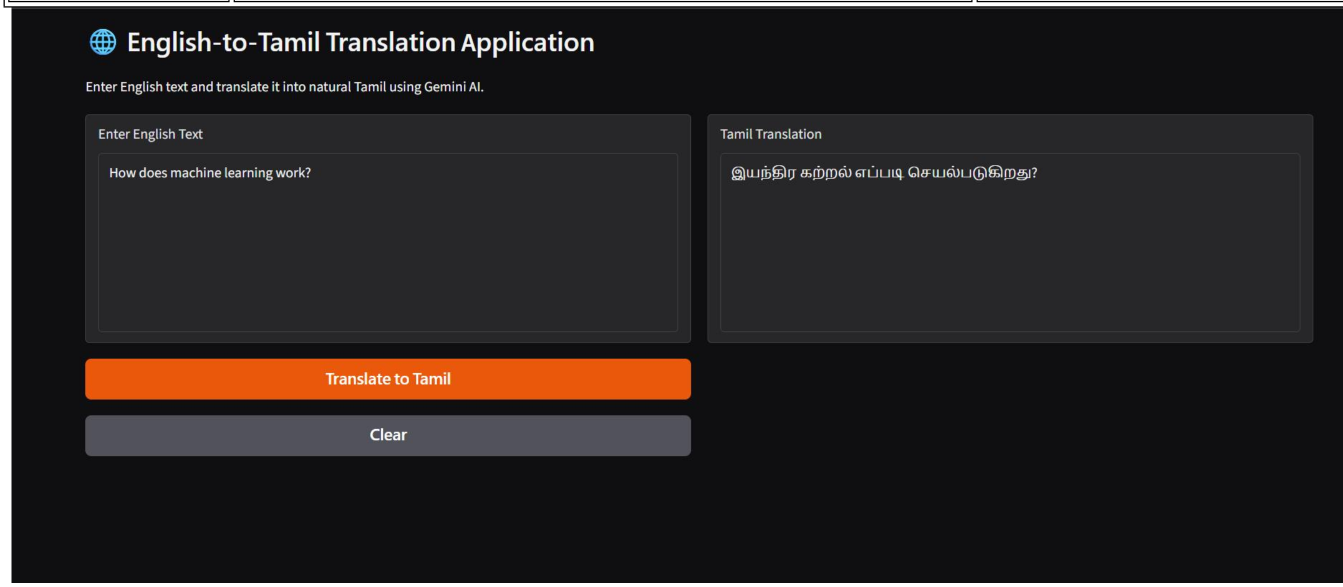
- Accepts English text as input.
- Translates English text into Tamil.
- Supports simple sentences.
- Supports questions and instructions.
- Supports longer sentences and paragraphs.
- Handles empty input.
- Handles invalid or very short input.
- Provides a simple Gradio user interface.
- Includes a Clear button to reset the application.



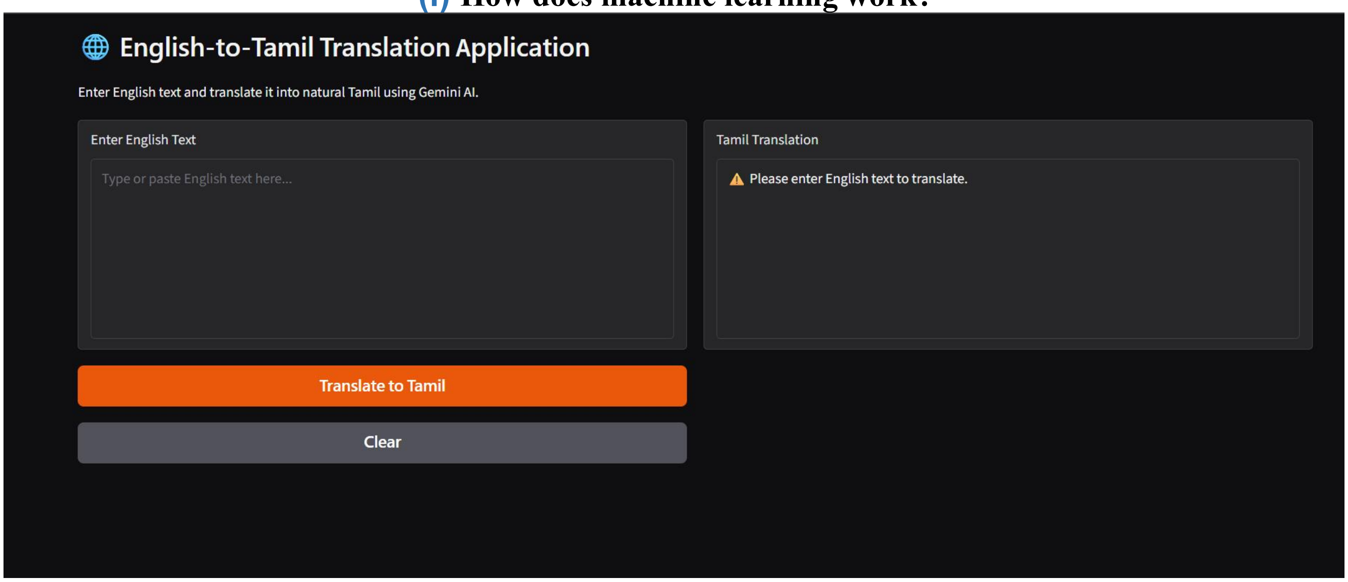
7. Test Cases and Results

Test Case	English Input	Expected Result
Simple Sentence	Artificial Intelligence is changing the world.	Tamil translation generated

Test Case	English Input	Expected Result
Question	How does machine learning work?	Tamil translation generated
Instruction	Please submit your assignment before tomorrow.	Tamil translation generated
Long Sentence	Generative AI is a technology that can create new content.	Tamil translation generated
Empty Input	No text entered	Warning message displayed
Very Short Input	A	Validation message displayed



(i) **How does machine learning work?**



(ii) No text entered

8. Invalid and Empty Input Handling

The application checks the input before sending it to the AI model.

For empty input, the system displays:

⚠ **Please enter English text to translate.**

For very short or invalid input, the system displays:

⚠ **Please enter a valid English sentence.**

This validation prevents invalid requests and improves the usability of the application.

9. Limitations and Future Improvements

Limitations

- Translation quality may vary depending on sentence complexity.
- Some technical terms or expressions may not have exact Tamil equivalents.
- The application requires an internet connection and Gemini API access.
- AI-generated translations may occasionally require human verification.

Future Improvements

- Add support for more languages.
- Add speech-to-text input.
- Add text-to-speech output.
- Support document and file translation.
- Add language selection for multiple source and target languages.

10. Conclusion

The English-to-Tamil Translation Application was successfully developed using Python, Gradio, and the Gemini AI model. The application accepts English text and generates a natural Tamil translation through an AI-powered translation process.

This project demonstrates how Generative AI and Large Language Models can be used to overcome language barriers and create simple, interactive machine translation applications.

40. Intelligent AI Assistant – Chatbot, Summarizer and Translator – Develop a complete AI application integrating chatbot, text summarization, and machine translation capabilities. The application should allow the user to select the required task, process the input appropriately, and display the result through a polished user interface. Demonstrate the system using multiple test cases and discuss the limitations and possible improvements.

1. Title

Intelligent AI Assistant Using Generative AI

2. Problem Statement and Objective

Users often require different AI applications for tasks such as asking questions, summarizing lengthy text, and translating content between languages. Using separate applications for each task can be inconvenient.

The objective of this project is to develop a complete **Intelligent AI Assistant** that integrates three capabilities: **Chatbot, Text Summarization, and English-to-Tamil Translation**. The user can select the required task, provide the input, and receive the AI-generated result through a single user-friendly interface.

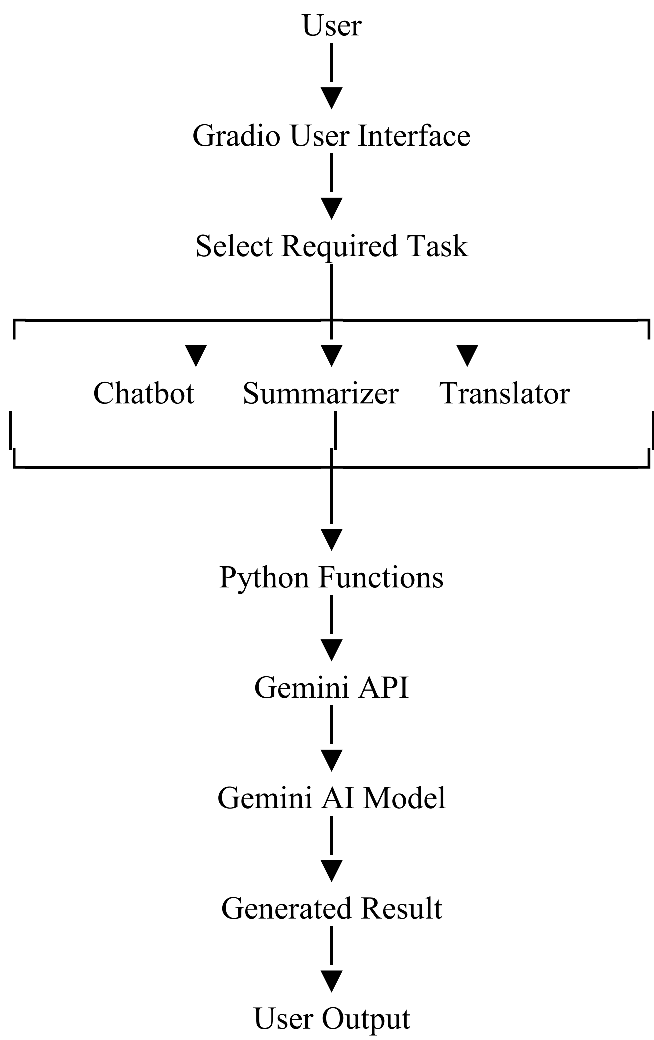
3. Technologies and AI Model Used

- **Programming Language:** Python
- **Development Platform:** Google Colab
- **User Interface:** Gradio
- **AI/LLM Model:** Gemini
- **Technique:** Generative AI, Text Generation, Text Summarization, and Machine Translation

Why Gemini?

Gemini was selected because it can perform multiple natural language processing tasks using a single Large Language Model. It can understand user questions, generate conversational responses, summarize long text, and translate English text into Tamil. This makes it suitable for developing an integrated multi-purpose AI application.

4. Application Architecture



The user selects one of the three available tasks through the Gradio interface. Based on the selected task, the corresponding Python function processes the input and sends an appropriate prompt to the Gemini AI model. The generated result is then displayed to the user.

5. Working Process

The application works as follows:

1. The user opens the Intelligent AI Assistant.
2. The user selects Chatbot, Summarizer, or Translator.
3. The user enters the required input.
4. The application validates the input.
5. The appropriate Python function is selected.
6. The request is sent to the Gemini AI model.

6. Application Features

Chatbot

The chatbot allows users to ask questions and receive AI-generated responses. It can provide explanations, answers, bullet points, or step-by-step information depending on the user's query.

Text Summarizer

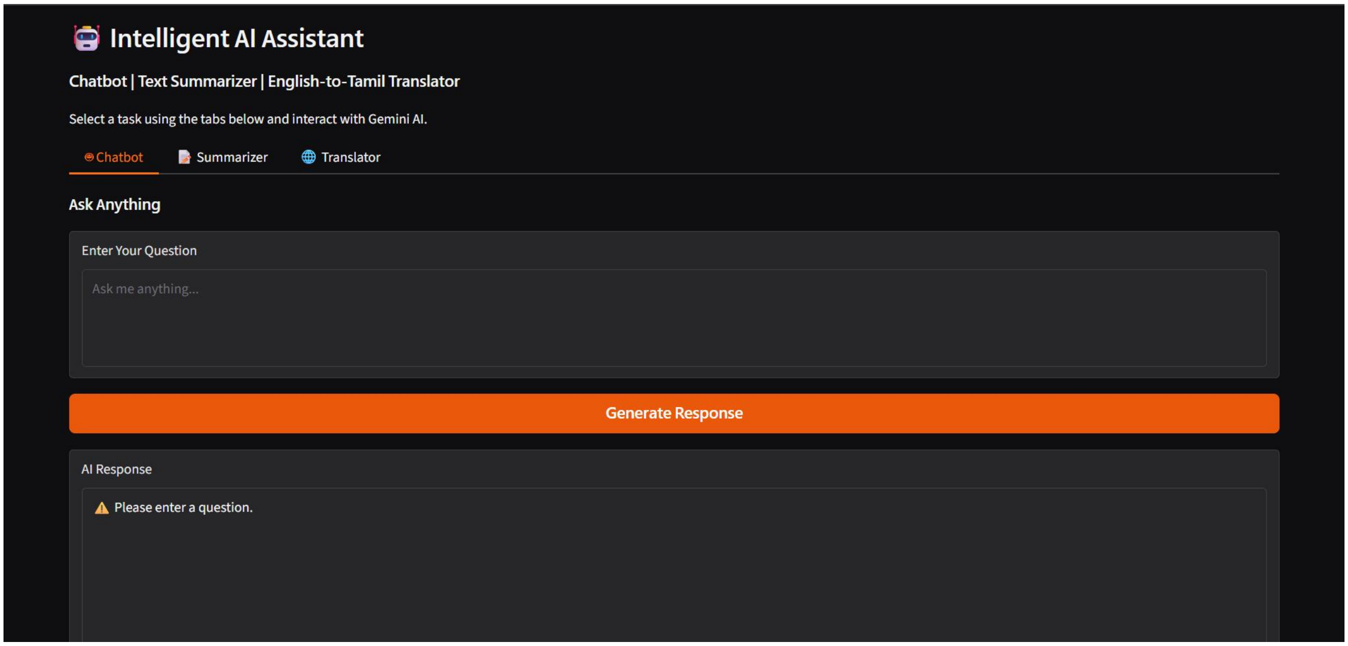
The summarizer accepts long text or articles and generates a concise summary. The user can select different summary lengths such as Short, Medium, or Detailed Summary.

English-to-Tamil Translator

The translator accepts English text and generates a natural Tamil translation while preserving the original meaning.

Other features include:

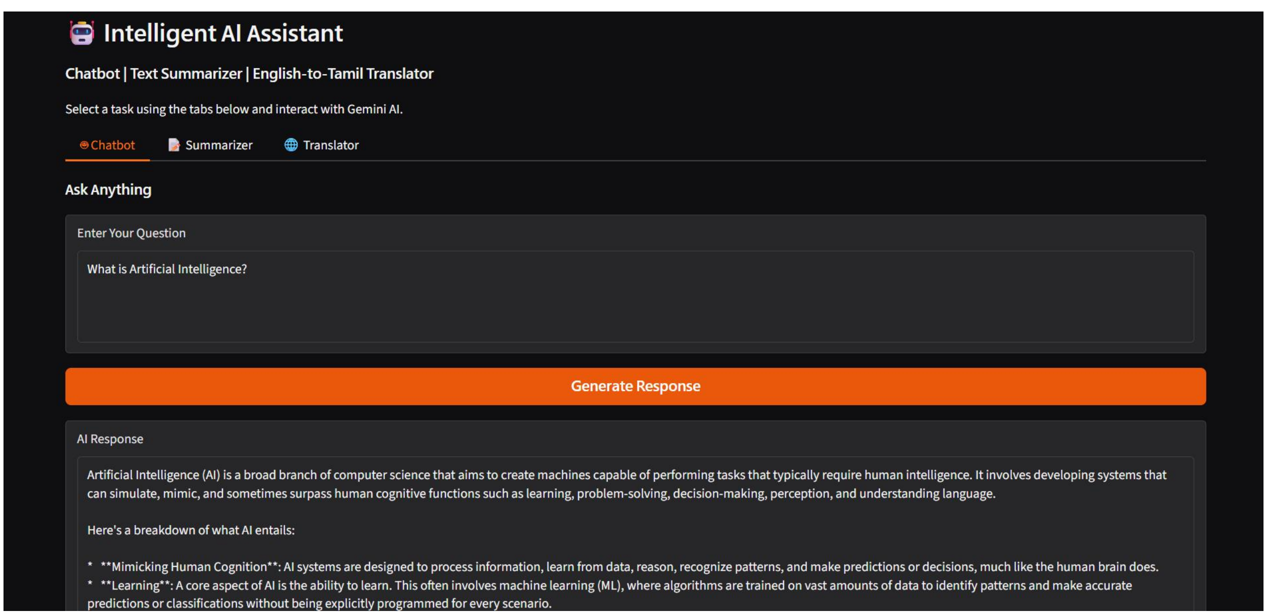
- User-friendly tab-based interface.
- Empty input validation.
- Error handling.
- Separate Clear buttons for each task.
- Integration of three AI capabilities into one application.



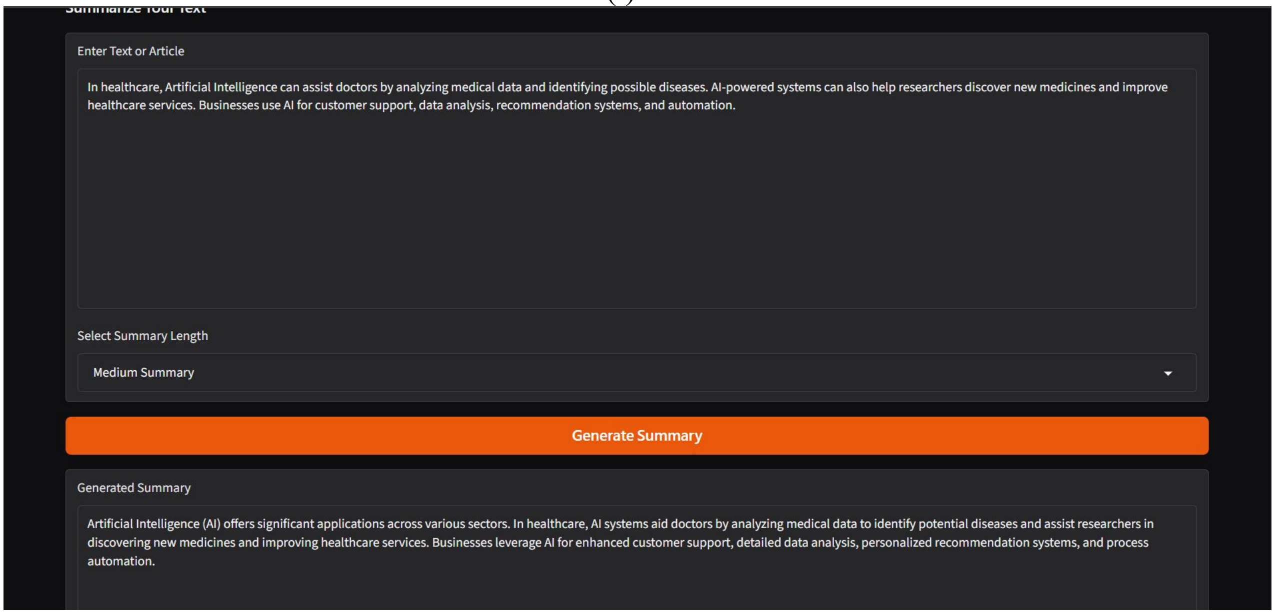
7. Test Cases and Results

Module	Test Input	Expected Result
Chatbot	What is Artificial Intelligence?	Relevant AI-generated answer
Chatbot	Explain machine learning in simple words.	Clear explanation generated
Summarizer	Long article about AI in Education	Concise summary generated
Summarizer	Same article with Short Summary	Shorter output generated
Translator	Artificial Intelligence is changing the world.	Tamil translation generated
Translator	Please submit your assignment tomorrow.	Tamil translation generated
Empty Input	No text entered	Warning message displayed

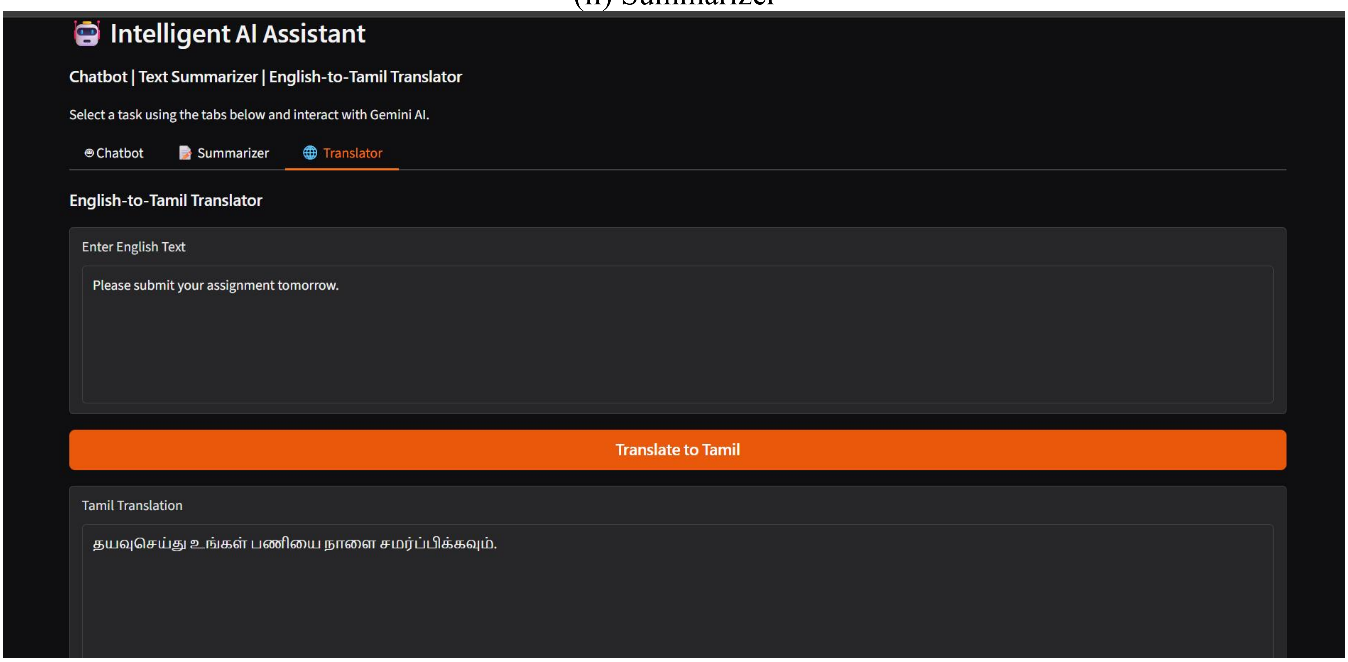
The application was tested using different inputs for all three modules. The chatbot generated relevant responses, the summarizer produced concise summaries based on the selected length, and the translator generated Tamil translations.



(i) Chatbot



(ii) Summarizer



(iii) Translator

8. Invalid and Empty Input Handling

Each module checks whether the user has entered valid input before processing the request.

Examples include:

- **Chatbot:** “Please enter a question.”
- **Summarizer:** “Please enter text to summarize.”
- **Translator:** “Please enter English text to translate.”

The summarizer also checks for very short text and requests the user to enter sufficient content for meaningful summarization.

9. Limitations and Future Improvements

Limitations

- AI responses may occasionally be inaccurate.
- The quality of summarization depends on the input text.
- Translation may not always perfectly handle complex expressions.
- The application requires internet access and Gemini API availability.
- The chatbot does not currently maintain long-term conversation memory.

Future Improvements

- Add chat history and conversation memory.
- Support more languages for translation.
- Add voice input and text-to-speech output.
- Support document and PDF summarization.
- Allow users to upload files directly.
- Add image and multimodal input support.
- Improve personalization based on user preferences.

10. Conclusion

The Intelligent AI Assistant was successfully developed using Python, Gradio, and the Gemini AI model. The application integrates three important AI capabilities: Chatbot, Text Summarization, and English-to-Tamil Translation.

The user can select the required task through a single interface and receive an appropriate AI-generated result. This project demonstrates how a Large Language Model can be integrated into a multi-purpose application to perform different natural language tasks efficiently and through a user-friendly interface.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Model/Technique Fit	30%	Correct, appropriate model/technique choice for the chosen application	Mostly appropriate choices made	Some inappropriate technique choices	Technique choice fundamentally flawed
Functional Completeness	25%	Application fully functional; solves a real use-case effectively	Mostly functional application	Partially functional application	Application is non-functional
Robustness & Reliability	20%	Handles edge cases; stable, reliable performance	Handles common cases well	Limited robustness to edge cases	Frequent failures during use
UI / Demo Polish	15%	Polished UI/demo with an engaging walkthrough	Clear demo, minor polish issues	Basic or rough demo	No usable demo presented
Design & Usage Documentation	10%	Comprehensive documentation of design & usage	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Model/Technique Fit (30%)	Functional Completeness (25%)	Robustness & Reliability (20%)	UI / Demo Polish (15%)	Design & Usage Documentation(10%)	Total Marks (100)
Q1						
Q2						
					Overall Total (100)	